Code Jam 2012 - Round 1C

Analysis: Box Factory

This problem is a variation of the Longest Common Subsequence problem, which is to find the longest string of characters which occurs as a subsequence of each of two strings (a string S is a subsequence of a string T if S occurs in T, possibly with more characters inserted between the elements of S.) In this case, the first "string" is the sequence of types of each box produced by the factory, and the second "string" is the sequence of types of each toy produced by the factory.

A dynamic programming algorithm for this problem is to find the maximum number of toys that could be placed in boxes using the first x boxes and the first y toys. Let this value be f[x][y]. Then f[x][y] is equal to the maximum of:

- f[x-1][y]
- f[x][y-1]
- f[x-1][y-1]+1

with the last case only applying if the x^{th} letter of the first string is equal to the y^{th} letter of the second string. These three cases correspond to the last action taken being to drop a box, to drop a toy, and to put a toy in a matching box, respectively.

Unfortunately, even though the number of runs of boxes and toys is small, the total number of boxes and toys can be very large, so this algorithm is not practical. But we can modify it so that f[x][y] will be the maximum number of toys that could be placed in boxes using the first x **runs** of boxes and the first y **runs** of toys. Now f[x][y] is the maximum of:

- f[x-1][y]
- f[x][y-1]
- f[a][b]+g(a,b,x,y), for all a<x, b<y

Similarly to before, the last case only applies if the type of box run x matches the type of toy run y. It corresponds to only using boxes of that type between runs a+1 and x, and toys of that type between runs b+1 and y. g(a,b,x,y) is the minimum of the number of those toys and the number of those boxes, which is the number of toys that can be placed in boxes in that range.

This algorithm is $O(n^4)$. Another improvement can reduce this again to $O(n^3)$, which we leave as an exercise to the reader!

Code Jam 2012 - Round 1C

Analysis: Diamond Inheritance

We are given a directed graph, and have to determine if there is a pair of nodes (X,Y) such that there are two or more paths from X to Y.

For each node, we do a depth-first search with that node as the root. If during the depth-first search we reach the same node twice, then we must have followed two different paths to get to that node from the root node, so we have found a pair (X,Y). Conversely, if there are two paths between X and Y, we will reach Y at least twice when doing a DFS from X. So if this algorithm finds no pair (X,Y), then none exists in the graph.

If there are V nodes and E edges in a graph, then a DFS is O(V+E) in general. But each DFS will never follow more than V edges, because after following that many edges, some node will have been reached twice, so we can stop at that point. Therefore this algorithm is $O(V^2)$.

We can also just use a variation of Floyd's algorithm, which is $O(V^3)$, but very simple to write. With only 1000 nodes in the graph, a fast implementation will finish within the time limit.

Code Jam 2012 - Round 1C

Analysis: Out of Gas

This was a hard problem to wrap your head around, as evidenced by the results. The final solution was, however, surprisingly simple (although the argumentation for it is not).

Let us first consider a single case, with a single acceleration value **a**, and an arbitrarily large **N**. Suppose some strategy S_1 takes you home in time **T**. Obviously $\mathbf{a}*\mathbf{T}^2 / 2 \geq \mathbf{D}$, as $\mathbf{a}*\mathbf{T}^2 / 2$ is the distance we travel if we start accelerating immediately and never brake.

We can propose an alternate strategy S_2: first stop and wait for time **T** - *sqrt*(2 **D**/ **A**), and then start accelerating full speed and never brake. This will also bring you home in time **T**.

We have to prove that if S_1 did not collide with the other car, neither will S_2. We prove this by checking that S_2 arrives at any point between the top of the hill and your home no earlier than S_1.

Assume S_1 was at some point **X** later than S_2. Notice that the speed of S_1 coming into **X** had to be larger than the speed of S_2 — otherwise even accelerating full speed will not let S_1 catch up with S_2. But it is impossible to achieve a faster speed at **X** than S_2: strategy S_2 accelerated all the way from the top of the hill to **X**.

So, now we only need to consider strategies such as S_1. We now just need to determine how long do we need to wait on the top of the hill.

One last thing to notice is that if the other car moves at constant speed between $\mathbf{x_i}$ and $\mathbf{x_{i+1}}$, and our strategy does not put us in front of the other car at $\mathbf{x_i}$ or at $\mathbf{x_{i+1}}$, then it does not intercept the other car in any intermediate point either. We know that because if we did intercept it in some intermediate point, it would mean that we were moving faster than the other car at the time of interception; and since we're accelerating, and the other car's speed is constant, we would end up in front of it at $\mathbf{x_{i+1}}$. Therefore passing the other car between $\mathbf{x_i}$ and $\mathbf{x_{i+1}}$ leads to a contradiction, and it must be the case that we don't pass it.

With this reasoning complete, we now have two possible algorithms. One is a binary search: To check if a given waiting time **T** is long enough, we just need to check all the **N** points at which the other car can change its speed. To achieve a precision of 10^{-6}, we will need around 40 iterations in the binary search. This means we solve each test case in O(**N A**) time, with the constant 40 hidden in the big-O notation — fast enough.

If we are worried about the constant factor of 40, we can also iterate over all points $\mathbf{x_i}$, and for each calculate the minimum waiting time needed not to intercept the other car at this point: $t_i - \mathtt{sqrt}(2x_i/a)$ for all i such that $x_i < D$ (you also have to include the moment where the other car reaches your house). This will make our solution run in O(**N A**) time without the pesky constant.